

Live Programmierung technoider Meditationsmusik mit analogem Synthesizer

Patrick Simmelbauer

Lukas Zahel

Phil Krause

20. Juli 2026

1 Einleitung

1.1 Idee und Konzept

Die Motivation für das vorliegende Projekt entstand im Rahmen eines Yoga-Klangbads, bei dem Instrumente wie Gongs und Klangschalen eingesetzt werden. Die dabei entstehenden Schwebungen erzeugen durch periodische Lautstärkeschwankungen eine kontinuierliche und fließende Klangwahrnehmung.

Ein strukturell verwandtes Prinzip lässt sich im Techno beobachten. Durch repetitive rhythmische Muster entstehen wiederkehrende zeitliche Strukturen. Während das Klangbad vor allem mit harmonischen Schwebungen und langsamen Klangveränderungen arbeitet, nutzt Techno hierfür primär rhythmische Wiederholung.

Ziel des Projekts ist es, diese beiden Konzepte in einer gemeinsamen Komposition zu verbinden. Als musikalische Orientierung dient dabei das Genre Ambient Techno, welches ambiente, flächige Klangtexturen mit repetitiven, technoiden Rhythmusstrukturen kombiniert [2].

Zur technischen Umsetzung wird das Live-Coding-Framework TidalCycles [11] eingesetzt. Dieses wird durch externe Hardware- und Softwarekomponenten wie einen analogen Synthesizer, ein MIDI-Keyboard sowie eine modulare Synthesizer-Software erweitert. Dadurch können unterschiedliche Klangquellen kombiniert und Möglichkeiten der interaktiven Klangerzeugung umgesetzt werden.

Der Songaufbau orientiert sich an einer Entwicklung von ruhigen, schwebenden Klangflächen hin zu komplexeren Ambient Techno Strukturen und greift am Ende wieder die ursprüngliche Klangbad Schwebung auf.

Das Musikstück stellt somit eine Verbindung zwischen harmonischer Schwebung und rhythmischer Repetition her. Unter Verwendung der stilistischen Mittel des Ambient Techno werden die ruhige, schwebende Klangästhetik des Klangbads und die rhythmischen Bewegungsstrukturen des Techno miteinander verbunden.

1.2 Techno

1989 markiert das Jahr, in dem das Musikgenre Techno erstmals eine breite öffentliche und kulturelle Wahrnehmung erlangte [8]. Techno ist ein überwiegend instrumentales Musikgenre, das sich im Laufe seiner Entwicklung stark ausdifferenziert hat [9]. Charakteristisch zeichnet es sich durch drei zentrale Merkmale aus: ein hohes Tempo ab circa 120 Beats pro Minute

(BPM) mit einer kontinuierlichen Bassdrum im Viervierteltakt („Four to the Floor“), die additive Überlagerung repetitiver perkussiver und melodischer Patterns und der Einsatz elektronisch erzeugter und häufig bewusst hart und technisch klingender Sounds [9]. Die Produktion verfolgt einen szenebasierten Do-It-Yourself-Ansatz (DIY) und wird mit Computer, elektronischen Instrumenten wie Drum Machines und Synthesizern sowie Samplern produziert [9].

1.3 Ambient

Ambient-Musik stellt ein Subgenre der elektronischen Musik dar, dessen Ursprünge sich in der ersten Hälfte des 20. Jahrhunderts verorten lassen [15]. Als Form atmosphärischer Hintergrundmusik konzentriert sich das Genre weniger auf melodische und rhythmische Entwicklung, als auf Klangfarben und die Schaffung eines bestimmten Klangraums [15].

Charakteristische Merkmale des Genres sind die bevorzugte Verwendung von Dur- oder pentatonischen Tonleitern, eine stark reduzierte, minimalistische Klangpalette und eine geringe harmonische Komplexität. Auf Gesang und perkussive Elemente wird weitestgehend verzichtet. Häufig werden organisch wirkende Klangfarben mit Naturaufnahmen kombiniert, welche durch einen ausgeprägten Hall-Effekt eine räumliche und atmosphärische Wirkung erhalten [4].

1.4 Ambient Techno

Ambient Techno bildet die Schnittmenge der beiden zuvor beschriebenen Genres. Es kombiniert die atmosphärischen, klangfarbenorientierten Texturen des Ambient mit den rhythmischen und produktionstechnischen Mitteln des Techno. Zu den prägenden KünstlerInnen zählen B12, Aphex Twin und Biosphere. Stilistisch vereint das Genre die melodischen und rhythmischen Ansätze von Techno mit den schwebenden, vielschichtigen, beatlosen und experimentellen Klanglandschaften des Ambient [2].

Das Genre legt den Fokus auf subtile klangliche Entwicklungen anstelle klassischer Drops und stellt emotionale Resonanz über strukturelle Komplexität. Die BPM-Zahl liegt dabei meist etwas unterhalb klassischer Techno-Strukturen und bewegt sich zwischen 90 und 125 BPM. Charakteristisch sind lange Wiederholungen mit graduellen Übergängen anstelle schneller Veränderungen [13].

Einen wichtigen Teil für einen atmosphärischen und tiefen Klang stellen Texturen dar. Diese können durch Field Recordings (Natur- oder Umgebungsgeräusche, die außerhalb des Tonstudios aufgenommen werden), bearbeitete Rauschanteile durch Filter und LFO-Modulationen, Vinyl-Knistern, sich verändernde harmonische Obertöne oder Pads erzeugt werden [13]. Pads stellen lang anhaltende polyphone Akkorde dar, welche typischerweise durch Synthesizer mit langsamen Attack- und Release-Zeiten, einer sich langsam verändernden Lautstärke sowie dem Einsatz von Hochpassfiltern erzeugt werden [7, 13]. Ein weiteres charakteristisches Merkmal ist die Illusion räumlicher Tiefe durch den Einsatz von Delay- und Halleffekten. Insgesamt entsteht dadurch eine ruhige und geheimnisvolle Atmosphäre [13].

Der Rhythmus im Ambient Techno ist subtil und minimal gestaltet. Weiche, tiefpassgefilterte Kickdrums mit zusätzlichem tieffrequentem Bassanteil sowie dezente Percussion unterstützen die atmosphärische Wirkung des Genres, ohne in den Vordergrund zu treten [13]. Eingesetzt werden dabei häufig techno-typische 808- und 909-Drumcomputer, reduzierte elektronische Klänge sowie Melodien in Moll und fremdartig klingende Samples [2].

Der Aufbau von Ambient Techno ist durch eine langsame, fließende Entwicklung der musikalischen Struktur geprägt. Ausgehend von wenigen atmosphärischen Elementen wird der Track durch das schrittweise Hinzufügen von Klängen und rhythmischen Komponenten zunehmend komplexer, bevor einzelne Elemente wieder reduziert werden und der Track schließlich ausklingt [13].

2 Hard- und Software-Aufbau

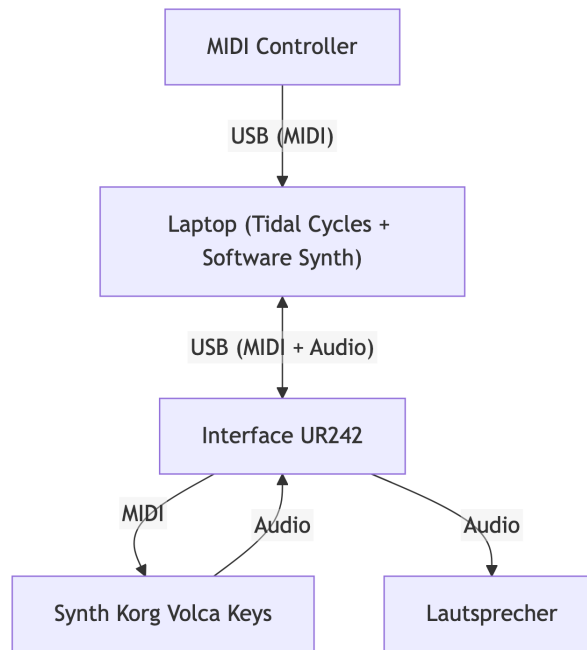


Abbildung 1: Der Hard- und Software-Aufbau des Projekts.

Im Projekt werden drei verschiedene Tonquellen genutzt. Dabei tauschen die Tonquellen untereinander Steuersignale aus und müssen am Ende zu einem einzelnen Tonsignal zusammengeführt werden. Da eine der Tonquellen ein externer Hardware-Synthesizer ist, reicht eine softwareseitige Verbindung nicht aus, und es ist zusätzliche Hardware notwendig. Eine Übersicht über die Verkabelung der Hardware-Komponenten ist in Abbildung 1 dargestellt.

2.1 Audio-Interface und Steuerung

2.1.1 Steinberg UR242

Das Herzstück des Aufbaus stellt das Audio- und MIDI-Interface *Steinberg UR242* dar. Dieses wird per USB-Anschluss mit dem Laptop verbunden und ermöglicht das Anbinden von externen Audio- und MIDI-Geräten. Das Interface kann dann im Betriebssystem als Audioausgabegerät genutzt werden. Das empfangene digitale Audiosignal wird im Interface in ein analoges Signal umgewandelt. Das analoge Ausgangssignal des Interfaces wird direkt mit dem Lautsprecher-Aufbau verbunden.

2.1.2 AKAI MPK mini

Zur Modulierung verschiedener Parameter des Software-Synthesizers wird ein *AKAI MPK mini* [1] verwendet. Dieses MIDI-Keyboards, das mit einer Klaviatur und verschiedenen Drehreglern ausgestattet ist, ist in der Lage, MIDI-Signale über eine USB-Schnittstelle zu versenden. In diesem Projekt kommen jedoch lediglich die Drehregler zum Einsatz. Diese versenden ein MIDI-Control-Change-Signal (MIDI-CC) (vgl. Abschnitt 3.4).

2.2 Tonquellen

Im folgenden Abschnitt werden die drei im Projekt verwendeten Tonquellen näher beschrieben. Dabei wird auf die Funktionsweise der Tonquellen eingegangen sowie erläutert, wie diese Instrumente im Projekt eingesetzt werden.

2.2.1 Korg Volca Keys

Der im Projekt verwendete externe Synthesizer ist der *Korg Volca Keys*. Sein MIDI-Eingang wird mit dem MIDI-Ausgang des Audio-Interfaces verbunden. Welche Noten zu welchem Zeitpunkt vom Synthesizer wiedergegeben werden sollen – oder anders ausgedrückt, welche Frequenzen die analogen Oszillatoren in ihm erzeugen sollen – wird über die MIDI-Schnittstelle gesteuert. Das dabei erzeugte Audiosignal wird zurück zum Interface geleitet und dort in ein digitales Signal umgewandelt, das der Laptop verarbeiten kann.

Der Korg Volca Keys ist ein analoger Synthesizer, der über drei Oszillatoren verfügt. Im Projekt wird er im Modus *UNISON* betrieben. In diesem Modus erzeugen die drei Oszillatoren des Synthesizers eine Sägezahn-Schwingung mit identischer Frequenz. Dies erzeugt einen volleren Klang als ein einzelner Oszillator. Zusätzlich kann durch den *DETUNE*-Regler die Frequenz eines der Oszillatoren leicht verändert werden. Auch dadurch entstehen Schwebungen (vgl. Abschnitt 3.1), die im Projekt als zentrales klangliches Mittel eingesetzt werden.

Der Synthesizer verfügt über einen Tiefpassfilter, der auch über den internen LFO moduliert werden kann. Ebenso sind die *ATTACK*-, *DECAY/RELEASE*- sowie *SUSTAIN*-Parameter des internen Hüllkurvengenerators über drei der verbauten Potentiometer einstellbar. Im Projekt wird zusätzlich der interne Delay-Effekt des Synthesizers genutzt. Dieser kann über die *TIME*- und *FEEDBACK*-Regler eingestellt werden.

2.2.2 datagraph

Neben dem Hardware-Synthesizer wird im Projekt ein Software-Synthesizer genutzt, welcher im Folgenden *datagraph* genannt wird [14]. Dieser ist über eine Browser-Oberfläche konfigurierbar und kann über die Web-MIDI-API [16] angesprochen werden. Das erzeugte Tonsignal wird über das Betriebssystem an das Audio-Interface geleitet.

Der Software-Synthesizer erzeugt Töne durch einen vom Nutzer programmierbaren Audio-Graph. Der Audio-Graph besteht aus Knoten, die miteinander verbunden werden können. Jeder Knoten hat eine bestimmte Funktion, wie z.B. die Erzeugung eines Tonsignals, die Modulation eines Signals oder die Addition von Signalen. Codebeispiel 1 zeigt eine einfache Implementierung eines Knotens, der zwei Eingangssignale addiert und das Ergebnis an einem Ausgang bereitstellt. Im Codebeispiel 2 ist zu sehen, wie dieser **Add**-Knoten dann programmatisch via Rust API genutzt werden kann. Hier werden zwei Sinus-Signale unterschiedlicher Frequenzen addiert und die ersten 10 generierten Sample-Werte ausgegeben.

```

1 pub struct Add;
2 impl Node<2, 1> for Add {
3     const INPUT_NAMES: [&'static str; 2] = ["input1", "input2"];
4     const OUTPUT_NAMES: [&'static str; 1] = ["output"];
5     const CATEGORY: &'static str = "math";
6     fn process(&mut self, input: [f32; 2], output: &mut [f32; 1]) {
7         output[0] = input[0] + input[1];
8     }
9     fn new(_: u32) -> Self {
10         Self
11     }
12 }

```

Codebeispiel 1: Implementierung eines Knotens zur Addition in der Software-Synthesizer-Umgebung.

```

1 let mut graph = Graph::new(44100);
2 let freq1 = graph.add_param(Frequency::from_hz(440.0).cv());
3 let freq2 = graph.add_param(Frequency::from_hz(880.0).cv());
4 let sin1 = graph.add::<Sin>();
5 let sin2 = graph.add::<Sin>();
6 let add = graph.add::<Add>();
7
8 // 440 Hz -> sin1
9 let _ = graph.connect_ports(&freq1.output_port(0).unwrap(), &sin1.input_port(0).unwrap());
10 // 880 Hz -> sin2
11 let _ = graph.connect_ports(&freq2.output_port(0).unwrap(), &sin2.input_port(0).unwrap());
12
13 // sin1 -> add
14 let _ = graph.connect_ports(&sin1.output_port(0).unwrap(), &add.input_port(0).unwrap());
15 // sin2 -> add
16 let _ = graph.connect_ports(&sin2.output_port(0).unwrap(), &add.input_port(1).unwrap());
17
18 let mut values = Vec::new();
19 for _ in 0..10 {
20     // Ausgangswert des add-Knotens auslesen und speichern
21     let value = *graph.port_value(&add.output_port(0).unwrap()).unwrap();
22     values.push(value);
23     graph.tick();
24 }
25 println!("{:?}", values); // [0.0, 0.07453336, 0.26173612, 0.4460045,
    0.625177, 0.7971648, 0.95998126, 1.1117709, 1.2508352, 1.3756568]

```

Codebeispiel 2: Beispielhafte Nutzung des Add-Knotens in der Software-Synthesizer-Umgebung.

Im Projekt wird jedoch nicht die Rust API direkt genutzt, sondern eine graphische Nutzeroberfläche mit welcher man Audio-Graphen dieser Art visuell erstellen kann. Abbildung 2 zeigt die visuelle Darstellung des im Projekt verwendeten Audio-Graphen in der Nutzeroberfläche.

Der gezeigte datagraph Patch nutzt 2 Sägezahn-Oszillatoren, die immer die Frequenz der zuletzt erhaltenen MIDI-Note wiedergeben. Die Frequenz eines der beiden Oszillatoren kann über ein MIDI-CC Signal (vgl. Abschnitt 3.4) angepasst werden, sodass auch hier ein

Schwebungseffekt erzeugt werden kann. Die beiden Oszillatoren werden dann gemischt und das gemischte Signal wird mit einer Hüllkurve multipliziert. Die Hüllkurve wird über einen **midigate**-Knoten gesteuert. Dieser extrahiert ein Gate Signal aus den erhaltenen „Note On“- und „Note Off“-MIDI-Signalen [12].

Um Elemente aus dem Techno wiederzuverwenden, wurde eine zusätzliche Hüllkurve definiert, welche ebenso über den **midigate**-Knoten gesteuert wird. Diese Hüllkurve wird dann genutzt, um die Filterfrequenz eines Tiefpassfilters anzupassen. Der Einfluss dieser Hüllkurve auf die Filterfrequenz kann durch einen MIDI-CC Parameter gesteuert werden. Dieses Verhalten gleicht dem Parameter *ENV MOD* des im Techno viel verwendeten Synthesizers *Roland TB-303*.

Zusätzlich gibt es noch die Möglichkeit, die Filterfrequenz sowie die Gesamtlautstärke direkt über weitere MIDI-CC Signale zu steuern.

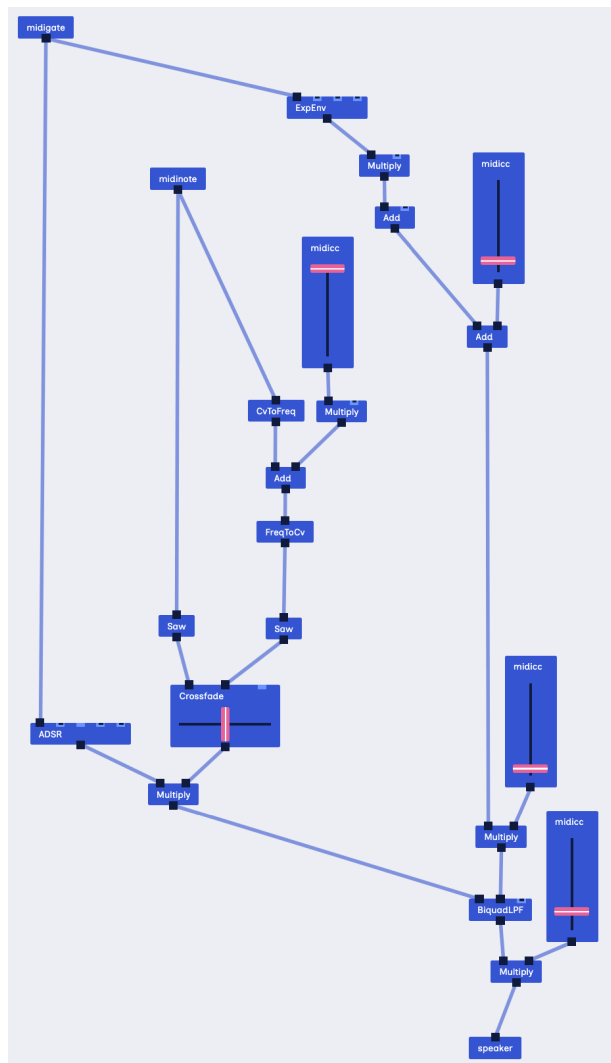


Abbildung 2: Der im Projekt genutzte Patch in der datagraph Nutzeroberfläche.

2.2.3 SuperCollider

Als dritte und letzte Tonquelle wird im Projekt *SuperCollider* [10] genutzt. SuperCollider dient als Backend für TidalCycles: TidalCycles steuert SuperCollider, SuperCollider erzeugt Audio- und MIDI-Signale und leitet diese an die jeweiligen Hard- und Software-Geräte weiter. Konkret erzeugt SuperCollider im Projekt die perkussiven Elemente und nutzt dafür Samples und Synthesizer, die mit der Installation von *TidalCycles* mitgeliefert werden.

3 Theorie

3.1 Schwebung

Zwei Schwingungen mit gleicher Amplitude und leicht unterschiedlichen Frequenzen werden vom menschlichen Ohr nicht als zwei getrennte Töne wahrgenommen; stattdessen hört man einen Ton einer Frequenz, die genau zwischen den beiden ursprünglichen Tönen liegt. Dieser wird jedoch kontinuierlich lauter und wieder leiser (vgl. Abbildung 3).

Dieser Effekt ist allgemein als Schwebung bekannt. Der Effekt ergibt sich direkt aus der Addition der beiden Schwingungen [5, S. 149ff].

$$A_{res}(t) = A \sin(2\pi f_1 t) + A \sin(2\pi f_2 t) \quad (1)$$

Es gilt:

$$\sin(a) + \sin(b) = 2 \sin\left(\frac{a+b}{2}\right) \cos\left(\frac{a-b}{2}\right) \quad (2)$$

Durch Einsetzen von Gleichung 2 in Gleichung 1 folgt dann:

$$\begin{aligned} A_{res}(t) &= 2A \cos\left(2\pi \frac{f_1 - f_2}{2} t\right) \sin\left(2\pi \frac{f_1 + f_2}{2} t\right) \\ &= 2A \cos(\pi f_{Schwebung} t) \sin(2\pi f_r t) \end{aligned} \quad (3)$$

Wobei $\frac{f_1+f_2}{2}$ die Frequenz des resultierenden Tones f_r beschreibt und $\frac{f_1-f_2}{2}$ die Frequenz mit der sich die Amplitude ändert. Da nur der Betrag der Amplitude interessant ist, schreiben wir $f_{Schwebung} = |f_1 - f_2|$ [5, S. 149].

Der Effekt der Schwebung kann zur Klangsynthese ausgenutzt werden, insbesondere bei der Verwendung gleicher Töne, von denen einer eine leichte Frequenzveränderung erfährt, ist die resultierende Schwebungsfrequenz direkt abhängig von der Änderung der Frequenz:

$$\begin{aligned} f_{Schwebung} &= |f_1 - f_2| \\ f_2 &= f_1 + f_{shift} \\ f_{Schwebung} &= |f_1 - (f_1 + f_{shift})| = |f_{shift}| \end{aligned} \quad (4)$$

Durch diese Eigenschaft können sehr gezielt Schwebungsfrequenzen erzeugt werden, die beispielsweise zum Takt der Musik passen.

3.2 Sägezahn-Schwingung

Gleichung 5 zeigt die Fourier-Zerlegung einer Sägezahn-Schwingung bekannt aus der Vorlesung und Übung.

$$f = -\frac{2}{\pi} \sum_{k \geq 1} \frac{(-1)^k}{k} \sin(kx) \quad (5)$$

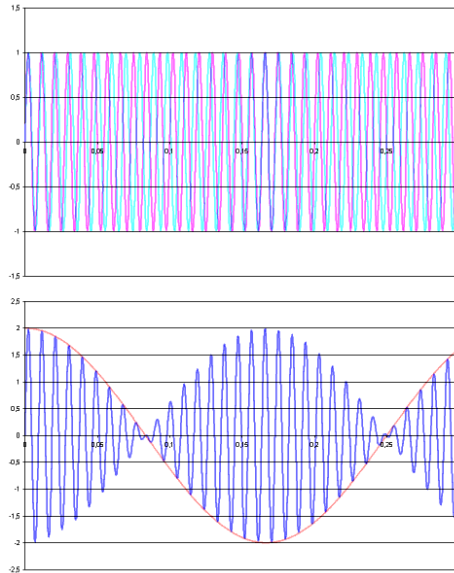


Abbildung 3: Veranschaulichung einer Schwebung. Die beiden oberen Signale mit leicht unterschiedlichen Frequenzen f_1 & f_2 (rot & blau) werden addiert und erzeugen das unten zu sehende resultierende Signal. Dieses besitzt die Frequenz $\frac{f_1+f_2}{2}$ und ändert seine Amplitude mit der Schwebungsfrequenz $|f_1 - f_2|$. Bildquelle: [3].

Aus der Gleichung folgt, dass eine Sägezahnswingung aus einer Grundfrequenz und deren ganzzahligen Vielfachen zusammengesetzt wird. Wird auf eine solche Schwingung nun ein Tiefpassfilter angewendet, dessen Filterfrequenz sich der Grundfrequenz f_0 der Sägezahnswingung annähert ($f_{filter} \rightarrow f_0$), so nähert sich das Signal einer reinen harmonischen (Sinus-)Schwingung an, da die höheren Vielfachen zunehmend gedämpft werden und im Idealfall nur die Grundfrequenz übrig bleibt. Aufgrund des unterschiedlichen Klangbilds einer Sägezahnswingung im Vergleich zu einer rein harmonischen Schwingung kann dieser Effekt ausgenutzt werden, um den Klang eines Synthesizers anzupassen.

Die durch einen Synthesizer erzeugte Sägezahnswingung wird hierbei durch einen Tiefpassfilter geleitet. Je höher die eingestellte Filterfrequenz ist, desto mehr gleicht die resultierende Schwingung dem Original, der Ton klingt also, typisch für Sägezahnswingungen, rau; $f_{filter} \rightarrow \infty$ wäre hierbei die perfekte Sägezahnswingung. Je mehr die Filterfrequenz nun der Grundfrequenz des erzeugten Tons angenähert wird, desto weicher klingt der resultierende Ton.

3.3 Delay

Der Delay Effekt spielt ein aufgenommenes Signal mit einer gewissen Zeitverzögerung immer wieder ab. Er ist dabei über zwei Parameter modifizierbar, nämlich die Zeitverzögerung mit der das Signal wiederholt wird und der Faktor um den das Signal bei jeder Wiederholung abgeschwächt wird. Das Signal wird hierbei bei modernen digitalen Delay-Systemen mithilfe eines Analog-Digital-Wandlers in einem kleinen Puffer gespeichert und anschließend wieder abgespielt (vgl. Boss DD-2 Digital Delay Pedal [6]).

Der Zeit-Parameter des Effekts gibt hierbei, gerade bei solchen digitalen Systemen, oft

lediglich an wie schnell das im Puffer gespeicherte Signal wieder abgespielt werden soll. Wie in der Vorlesung gezeigt, kann dadurch ein zusätzlicher Effekt erzeugt werden. Durch Veränderung des Zeit-Parameters während ein Signal abgespielt wird, kann die Tonhöhe verändert werden.

Das schnellere, beziehungsweise langsamere, Durchlaufen des Puffers sorgt dafür, dass die digital gespeicherten Werte zu einem analogen Signal mit einer vom original Signal unterschiedlichen Frequenz zurückgewandelt werden.

3.4 MIDI-Control Change (MIDI-CC)

Eine *MIDI Control Change* Nachricht besteht aus einer Control Nummer (ganze Zahl zwischen 0 und 127), sowie einem Wert (ebenfalls ganze Zahl zwischen 0 und 127). Mithilfe dieser Nachrichten können verschiedene Parameter eingestellt werden. Bestimmte Control Nummern sind für die Einstellung gewisser Parameter vorgesehen [12]. Es kann also beispielsweise über Control Nummer 7 die Lautstärke eines Kanals angepasst werden.

4 Ablauf des Live-Konzerts

Das gesamte Konzert findet in einer Geschwindigkeit von 128 Schlägen pro Minute statt. Dies wird erreicht, indem zu Beginn einmal der Befehl `setcps` ($128/60/4$) ausgeführt wird.

4.1 Klangbad mit Gong Schwebung

Das Konzert beginnt mit dem Klang von zwei Klangschalen des gleichen Tones. Anschließend wird der Klang einer der beiden Klangschalen mit dem `fshift`-Effekt von `TidalCycles` modifiziert. Durch die leichte Frequenzverschiebung eines der beiden Töne entsteht eine Schwebung (siehe Abschnitt 3.1). Da `fshift` alle Frequenzanteile um denselben konstanten Betrag verschiebt ($f_i \mapsto f_i + \Delta f$), schwebt jedes Partialpaar des obertonreichen `supergong`-Samples mit derselben Frequenz Δf – anders als bei einem multiplikativen Pitch-Shift, bei dem jede Harmonische eine andere Schwebungsfrequenz erhielte. Dadurch entsteht trotz des inharmonischen Klangs eine einzige, klar definierte Schwebung. Der Effekt ist dabei so eingestellt, dass eine Frequenzverschiebung von $k \cdot \frac{128}{60}$ mit $k \in \mathbb{N}$ stattfindet. Dadurch wird eine Schwebungsfrequenz erzeugt, die ein ganzzahliges Vielfaches der Schlagrate (Schläge pro Sekunde) beträgt. Die Wellenbäuche bzw. -knoten der Schwebung passen damit zum Puls der Musik. Die Umsetzung in `TidalCycles` ist in Codebeispiel 3 zu sehen. Die Funktion `gong` erzeugt hierbei zwei Töne, von welchen einer mit dem `fshift`-Effekt modifiziert wird. Der Parameter `shift` entspricht dabei dem oben genannten k und kann beliebig verändert werden, um die gewünschte Schwebungsfrequenz zu erzeugen.

```

1 let bpm = 128
2 setcps (bpm/60/4)
3
4 gong shift = stack [ slow 2 $ n "f3" # s "supergong"
5                      , slow 2 $ n "f3" # s "supergong" # fshift (shift * bpm/60)
6                      ]
7
8 d1 $ gong 2 # gain 0.6

```

Codebeispiel 3: Erzeugung von `supergong`-Tönen mit Parametrisierung der Schwebungsfrequenz.

4.2 Percussions

Nun werden geschlossene Highhats hinzugefügt, welche mit einem Tiefpassfilter moduliert werden. Die Filterfrequenz wird hierbei mit einem Sinus der halben Schwebungsfrequenz geändert. Über 32 Cycles werden die Highhats langsam lauter (vgl. Codebeispiel 4).

```
1 d6 $ fast 1 $ sound "superhat*8"
2   # lpf (fast 2 $ range 1000 "1500" $ sine)
3   # gain 0
4   # room 0.2
5
6 xfadeIn 6 32 $ fast 1 $ sound "superhat*8"
7   # lpf (fast 2 $ range 1000 "1500" $ sine)
8   # gain 0.7
9   # room 0.2
```

Codebeispiel 4: fadeIn der Highhats

4.3 Bass

Jetzt wird datagraph als Tonquelle zugeschaltet und erhält die Noten für einen tiefen Bass-Sound von TidalCycles. Dieser wurde mithilfe des Software-Synthesizers, der in Abschnitt 2.2.2 beschrieben wird, modular zusammengebaut, besteht aus zwei Sägezahn-Schwingungen und besitzt verschiedene Parameter, welche über Drehknöpfe des MIDI-Keyboards angepasst werden können (vgl. Abschnitt 3.4).

Die vier einstellbaren Parameter sind eine Frequenz-Verschiebung einer der beiden Sägezahn-Schwingungen (ähnlich wie bei den Klangschalen um $k \cdot \frac{128}{60}$), um eine gezielte Schwebung zu erzeugen, ein Tiefpassfilter, mit dem der Klang zwischen Sägezahn und harmonischer Schwingung verändert werden kann (vgl. Abschnitt 3.2), ein Lautstärke-Regler und eine Einstellung, mit welcher ein Filter, der wiederum an den Attack-Wert der beiden Schwingungen gebunden ist, eingestellt werden kann.

Der Bass wird zunächst mit einem Filterwert nahe der Grundfrequenz des Tons und einer Schwebung gleich der der Klangschalen verwendet. Nun wird ein Filter auf die Klangschalen angewendet, bevor diese ganz entfernt werden.

4.4 Rhythmus und Synth

Verschiedene Schlagelemente der typischen 808-Drum-Samples (vgl. Abschnitt 1.4, Codebeispiel 5) werden zugeschaltet sowie der *Korg Volca Keys*-Synthesizer. Dieser spielt zunächst jedoch nur einzelne Töne zu Beginn jedes zweiten Taktes und keine Melodie (vgl. Codebeispiel 6). Durch Einstellen eines hohen Delay-Feedbacks und Veränderung der Zeit-Komponente wird eine Art Tremolo mit schwankender Tonhöhe erzeugt (siehe Abschnitt 3.3).

```
1 d7 $ slow 1 $ "[808bd <[808sd 808oh] [808oh 808oh*2]>]*2" # gain 0.7 # lpf
   2500
```

Codebeispiel 5: Drum Pattern

```
1 d8 $ slow 2 $ s "mydevice" # n "<-2, 1, 5>"
```

Codebeispiel 6: MIDI Weiterleitung an analogen Synth

4.5 Pad

Ein Pad wird aus drei zyklisch wechselnden Akkorden innerhalb der pentatonischen `minPent`-Skala erstellt (vgl. Codebeispiel 7, vgl. typische Skala für Ambient aus Abschnitt 1.3). Das Pad erzeugt eine Hintergrundtextur und fügt gleichzeitig eine melodische Ebene hinzu. Eine lange Attack- und Release-Zeit in Kombination mit einer hohen Sustain-Skalierung sorgt dafür, dass die Akkorde ohne hörbaren Einsatzpunkt einschwingen und weich ineinander übergehen. In Kombination mit `legato` überlappen sich aufeinanderfolgende Akkorde zeitlich nahezu vollständig, sodass der perkussive Charakter eines klassischen Akkordanschlags vermieden wird und ein durchgehender Klang entsteht. Ein stark ausgeprägter Reverb sowie eine sich kontinuierlich verändernde Lautstärke und Tiefpassfilter-Grenzfrequenz verleihen dem Pad abschließend die räumliche Tiefe und Bewegung für den sphärischen Gesamtklang.

```
1 d3 $ slow 2 $ n ( scale "minPent" "<[-2, 1, 5] [1, 3, 4, 7] [0, 3, 7]>" ) # s "
    superzow"
2   # attack 1
3   # release 1
4   # legato 0.99
5   # lpf (slow 20 $ range 200 "1000" $ sine)
6   # amp (slow 10 $ range 0.3 0.9 $ sine)
7   # room 0.9 # size 0.95
8   # gain 0.7
```

Codebeispiel 7: Erzeugung Hintergrund Pad

4.6 Randomisierung

Im weiteren Verlauf des Stücks wird der analoge Synthesizer durch eine randomisierte Notenfolge angesteuert, um die Komplexität gezielt zu manipulieren (vgl. Codebeispiel 8). Zusätzlich lässt sich mittels `degradeBy` die Anzahl der gespielten Noten verändern. Die Funktion entfernt jedes Event mit einer übergebenen Wahrscheinlichkeit zwischen 0 und 1. Über den Verlauf des Stücks werden dabei unterschiedliche Werte eingesetzt, um die Komplexität gezielt zu erhöhen oder zu reduzieren. Bei einem Wert von 0.8 wird jedes Event der randomisierten Noten mit einer Wahrscheinlichkeit von 80 % entfernt, wodurch deutlich weniger Noten gespielt werden. Bei einem Wert von 0.3 bleibt hingegen der Großteil der Events erhalten, was im Hauptteil des Stücks für eine höhere Komplexität genutzt wird.

```
1 d8 $ slow 2 $ degradeBy 0.3 $ n (scale "minPent" (fmap fromIntegral $ randrun
    8)) # s "mydevice"
```

Codebeispiel 8: Randomisierung der Synth Noten

4.7 Ausklang

Das Musikstück endet, indem es wieder zum Gong zurückkehrt, mit dem es begonnen hat. Die übrigen Texturen werden währenddessen nach und nach ausgeblendet, bis schließlich nur noch der Gong zu hören ist und auch dieser langsam verstummt. Damit schließt sich der eingangs beschriebene Spannungsbogen: Das Stück kehrt zu den Schwebungen zurück, die während der gesamten Spieldauer sein klangliches Fundament bildeten.

4.8 Techniken für graduelle Übergänge

Zusammenfassend werden im Stück drei Techniken für graduelle Übergänge genutzt: `xfadeIn` blendet ganze Klangelemente über die Lautstärke ein und aus (z. B. `xfadeIn 6 32 $ ... # gain 0.7`), `interpolateIn` überführt die Parameter einer laufenden Stimme kontinuierlich in neue Zielwerte (z. B. `interpolateIn 1 32 $ gong 2 # gain 0.5`), und `range` in Kombination mit `sine` moduliert einen beliebigen Parameter fortlaufend zwischen zwei Grenzwerten (z. B. `lpf (range 1000 "5000"$ sine)`). Letzteres lässt sich auch auf die Schwebungsfrequenz des Gongs anwenden (vgl. Codebeispiel 9). Alle drei Techniken werden eingesetzt um abrupte Sprünge zugunsten kontinuierlicher Übergänge zu vermeiden. Dieses Prinzip entspricht dem stilistischen Mittel von Ambient Techno aus Abschnitt 1.4.

```
1 d1 $ gong (slow 4 $ range 2 4 $ sine) # gain 0.6
```

Codebeispiel 9: Modulation der Schwebungsfrequenz mittels `range` und `sine`

Literatur

- [1] Akai Professional. *MPK mini mk3*. Akai Professional. 2020. URL: <https://www.akaipro.com/mpk-mini-mk3/> (besucht am 19.07.2026).
- [2] NETAKTION LLC ALLMUSIC. *Ambient Techno Music Style Overview*. 2026. URL: <https://www.allmusic.com/style/ambient-techno-ma0000012032> (besucht am 18.07.2026).
- [3] Barak Sh. *Beat.png*. Wikimedia Commons. 2008. URL: <https://commons.wikimedia.org/wiki/File:Beat.png> (besucht am 19.07.2026).
- [4] George Burdon. „Immunological atmospheres: Ambient music and the design of self-experience“. en. In: *cultural geographies* 30.4 (Okt. 2023), S. 555–568. ISSN: 1474-4740, 1477-0881. DOI: 10.1177/14744740231167604. URL: <https://journals.sagepub.com/doi/10.1177/14744740231167604> (besucht am 17.07.2026).
- [5] Christian Gerthsen. *Gerthsen Physik*. Hrsg. von Dieter Meschede. 22. Aufl. Berlin: Springer, 2004. ISBN: 3-540-02622-3.
- [6] Hobby-Hour.com. *Speed of Sound Delay Calculator*. Hobby-Hour. n.d. URL: <https://www.hobby-hour.com/electronics/s/dd2-delay.php> (besucht am 19.07.2026).
- [7] Baby Audio Inc. *Ambient Pads: 5 Synth Ideas With Physical Modeling*. 2026. URL: <https://babyaud.io/blog/ambient-pads> (besucht am 18.07.2026).
- [8] Max Ischebeck. „Das Subjekt im Klang“. de. In: *Das Subjekt im Klang*. Series Title: Auditive Vergesellschaftungen Hörsinn - Audiotechnik - Musikerleben. Wiesbaden: Springer Fachmedien Wiesbaden, 2025, S. 461–506. ISBN: 978-3-658-47107-1 978-3-658-47108-8. DOI: 10.1007/978-3-658-47108-8_6. URL: https://link.springer.com/10.1007/978-3-658-47108-8_6 (besucht am 17.07.2026).
- [9] Timor Kaul. „Techno“. de. In: *Handbuch Popkultur*. Hrsg. von Thomas Hecken und Marcus S. Kleiner. Stuttgart: J.B. Metzler, 2017, S. 106–110. ISBN: 978-3-476-02677-4 978-3-476-05601-6. DOI: 10.1007/978-3-476-05601-6_19. URL: http://link.springer.com/10.1007/978-3-476-05601-6_19 (besucht am 17.07.2026).

- [10] James McCartney. „SuperCollider: a new real time synthesis language“. In: *Proceedings of the International Computer Music Conference*. 1996.
- [11] Alex McLean und Geraint Wiggins. „Tidal–pattern language for the live coding of music“. In: *Proceedings of the 7th sound and music computing conference*. 2010, S. 331–334.
- [12] Bob Moog. „MIDI“. In: *Journal of Audio Engineering Society* 34.5 (1986). URL: <https://moogfoundation.org/wp-content/uploads/5267-1.pdf> (besucht am 20.07.2026).
- [13] Samplesound. *Ambient Techno: Crafting Textures and Pads for Immersive Soundscapes*. 2026. URL: <https://www.samplesoundmusic.com/blogs/news/ambient-techno-crafting-textures-and-pads-for-immersive-soundscapes> (besucht am 18.07.2026).
- [14] Patrick Simmelbauer. *datagraph: A browser-based, node-graph editor for building audio signal chains*. <https://github.com/patsimm/datagraph>. 2026.
- [15] Iliana Velescu. „Ambient music“. In: *Education, Research, Creation* 1 (2015), S. 112–117. URL: <https://www.ceeol.com/search/article-detail?id=497794>.
- [16] Michael Wilson und Chris Wilson. *Web MIDI API*. W3C Working Draft. W3C, Jan. 2025. URL: <https://www.w3.org/TR/2025/WD-webmidi-20250121/>.